

cis112

Binary Search Trees

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-05-03

Content

- 1 Motivation
- 2 Definitions
- 3 Algorithms
- 4 Implementation
- 5 Applications



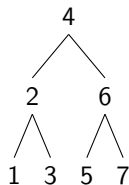
Motivation

Game of secret number

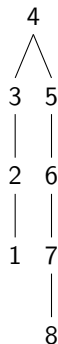
Game.

- Player-A selects a secret number in the range of $\{1, 2, \dots, 8\}$.
- Game repeats till the secret is found.
 - Player-B tries a number.
 - Player-A should answer as “found”, or “smaller” or “larger”.

Q. What is the best strategy if the range becomes large such as $\{1, 2, \dots, 1000, 000\}$.



This strategy finds the secret faster.



This strategy is slower.

Definitions

Binary Search Tree

Definition

Binary search tree property :

Let x be a node in a binary tree.

- 1 If y is a node in the **left subtree** of x , then $y.data \leq x.data$.
- 2 If y is a node in the **right subtree** of x , then $y.data \geq x.data$.

A binary tree with the binary-search-tree property is called a **binary search tree**.

Example



2 is in the left subtree of 5 since $2 \leq 5$.
 2 is in the left subtree of 6 since $2 \leq 6$.
 8 is in the right subtree of 7 since $8 \geq 7$.
 8 is in the right subtree of 6 since $8 \geq 6$.

[2]

[1], [2], [3], [4], [5], [6], [7]

Algorithms

Node

Algorithm 1: Node for Binary Search Tree

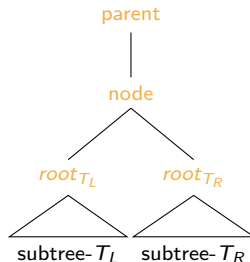
```
1 begin
2   data
3   parent           // reference to the parent
4   left             // reference to the left child
5   right            // reference to the right child
```

[[2]]

Warning. data is comparable:

Comparison of a.data and b.data for Nodes, a and b, is defined as

$$a.data.compareTo(b.data) = \begin{cases} < 0, & a.data < b.data, \\ 0, & a.data == b.data, \\ > 0, & a.data > b.data. \end{cases}$$



Algorithms

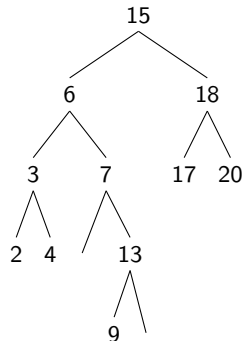
Tree Traversal

Inorder Tree Traversal

Function InorderTreeWalk(x) [\[\[2\]\]](#)

```

1 if x ≠ NIL then
2   InorderTreeWalk(x.left)
3   print x.data
4   InorderTreeWalk(x.right)
  
```



Output of InorderTreeWalk (15)
2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20.

Algorithms

Searching

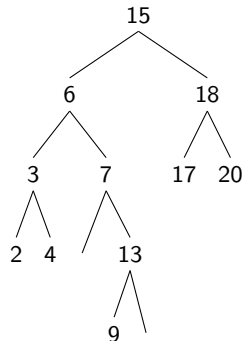
Tree Search

Function TreeSearch(x, k) [[2]]

```

1 if  $x == \text{NIL}$  or  $\text{key} == x.\text{key}$  then
2   | return  $x$ 
3 if  $k < x.\text{key}$  then
4   | return TreeSearch(  $x.\text{left}, k$ )
5 else
6   | return TreeSearch(  $x.\text{right}, k$ )

```



Trace of TreeSearch (15, 14)
TreeSearch (15,14)
TreeSearch (6,14)
TreeSearch (7,14)
TreeSearch (13,14)
TreeSearch (NIL,14)

Trace of TreeSearch (15, 13)
TreeSearch (15,13)
TreeSearch (6,13)
TreeSearch (7,13)
TreeSearch (13,13)

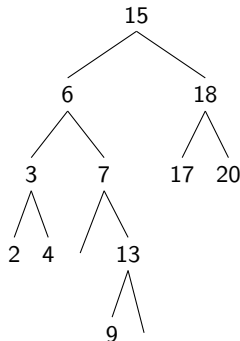
Iterative Tree Search

Function IterativeTreeSearch(x, k) [[2]]

```

1 while  $x \neq \text{NIL}$  and  $k \neq x.\text{key}$  do
2   if  $k < x.\text{key}$  then
3      $x = x.\text{left}$ 
4   else
5      $x = x.\text{right}$ 
6 return  $x$ 

```



Trace of IterativeTreeSearch (15, 13)

x : 15 // lines: 1-5

x : 6 // lines: 1-5

x : 7 // lines: 1-5

x : 13 // lines: 1-5

Algorithms

Minimum, Maximum, Successor, Predecessor

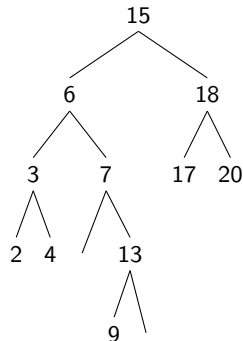
Tree Minimum

Function TreeMinimum(x) [12]

```

1 while x.left ≠ NIL do
2   | x = x.left
3 return x

```



- Q. TreeMinimum (6)?
- Q. TreeMinimum (7)?
- Q. TreeMinimum (18)?

Trace of TreeMinimum (15)
15 // lines: 1-2
6 // lines: 1-2
3 // lines: 1-2
2 // lines: 1-2

Tree Maximum

Function TreeMaximum(x) [[2]]

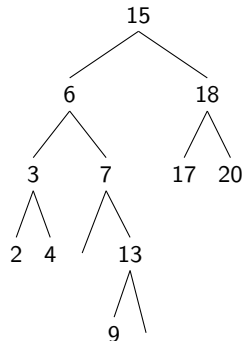
```

1 while x.right ≠ NIL do
2   | x = x.right
3 return x

```

Q. TreeMaximum (3)?

Q. TreeMaximum (4)?



Trace of TreeMaximum (6)

6 // lines: 1-2
7 // lines: 1-2
13 // lines: 1-2

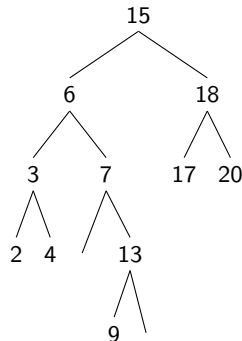
Trace of TreeMaximum (15)

15 // lines: 1-2
18 // lines: 1-2
20 // lines: 1-2

Tree Successor

Function TreeSuccessor(x) [12]

```
1 if x.right ≠ NIL then
    // leftmost node in right subtree
2     return TreeMinimum(x.right)
3 else
    // find the lowest ancestor of x,
    // whose left child is an ancestor of x
4     y = x.parent
5     while y ≠ NIL and x == y.right do
6         x = y
7         y = y.parent
8 return y
```



Trace of TreeSuccessor (13)

x:13, y:7 // lines: 4-7

x:7, y:6 // lines: 4-7

x:6, y:15 // lines: 4-7

Trace of TreeSuccessor(15)

15 // line: 2

18 // line: 2

17 // line: 2

Algorithms

Insertion, Deletion

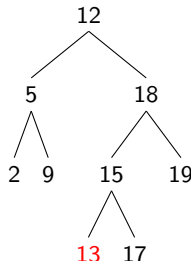
Tree Insert

Function TreeInsert(T, z) [2]

```

1  x = T.root           // node being compared with z
2  y = NIL              // will be parent of z
3  while x ≠ NIL do
4      // descend until reaching a leaf
5      y = x
6      if z.key < x.key then
7          x = x.left
8      else
9          x = x.right
10 z.parent = y // found the location- insert z with parent y
11 if y == NIL then
12     T.root = z // tree was empty
13 else if z.key < y.key then
14     y.left = z
15 else
16     y.right = z

```



Trace of TreeInsert (T, 13)

x:12, y:NIL // lines: 1-2
 x:18, y:12 // lines: 3-8
 x:15, y:18 // lines 3-8
 x:NIL, y:15 // lines: 3-8
 15.left=13 // lines: 10-15

Deleting node z

Deleting a node z , in **blue**, from a binary search tree. Node z may be the root, a left child of node q , or a right child of q . The node that will replace node z in its position in the tree is colored **orange**.

a) Node z has **no left child**.

a)

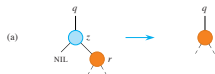


Deleting node z

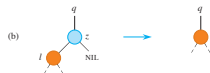
Deleting a node z , in **blue**, from a binary search tree. Node z may be the root, a left child of node q , or a right child of q . The node that will replace node z in its position in the tree is colored **orange**.

- a) Node z has **no left child**.
- b) Node z has a left child ℓ but **no right child**.

a)



b)



Deleting node z

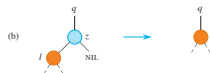
Deleting a node z , in **blue**, from a binary search tree. Node z may be the root, a left child of node q , or a right child of q . The node that will replace node z in its position in the tree is colored **orange**.

- a) Node z has **no left child**.
- b) Node z has a left child ℓ but **no right child**.
- c) Node z has **two children**. Its left child is node ℓ , its right child is its successor y (which has **no left child**), and y 's right child is node x .

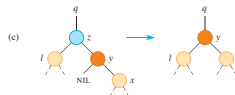
a)



b)



c)

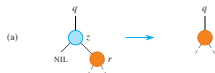


Deleting node z

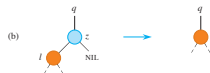
Deleting a node z , in blue, from a binary search tree. Node z may be the root, a left child of node q , or a right child of q . The node that will replace node z in its position in the tree is colored orange.

- Node z has no left child.
- Node z has a left child ℓ but no right child.
- Node z has two children. Its left child is node ℓ , its right child is its successor y (which has no left child), and y 's right child is node x .
- Node has two children (left child ℓ and right child r), and its successor $y \neq r$ lies within the subtree rooted at r .

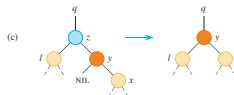
a)



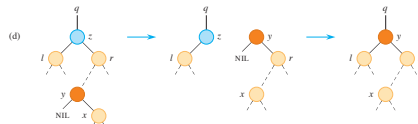
b)



c)



d)



Transplant

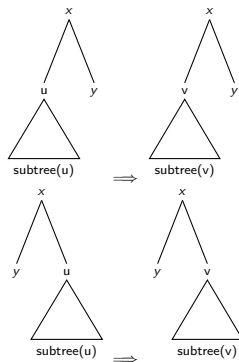
Function Transplant(T, u, v) [2]

```

1 if  $u.p == \text{NIL}$  then
2   |  $T.\text{root} = v$ 
3 else if  $u == u.p.\text{left}$  then
4   |  $u.p.\text{left} = v$ 
5 else
6   |  $u.p.\text{right} = v$ 
7 if  $v \neq \text{NIL}$  then
8   |  $v.p = u.p$ 

```

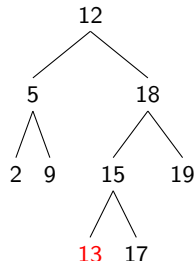
TRANSPLANT replaces one subtree as a child of its parent with another subtree. When TRANSPLANT replaces the subtree rooted at node u with the subtree rooted at node v , node u 's parent becomes node v 's parent, and u 's parent ends up having v as its appropriate child.



Tree Delete

Function TreeDelete(T, z) [[2]]

```
1 if z.left == NIL then
2   Transplant (T, z, z.right)      // replace z by its right child
3 else if z.right == NIL then
4   Transplant (T, z, z.left)      // replace z by its left child
5 else
6   y = TreeMinimum (z.right)      // y is z's successor
7   if y ≠ z.right then            // is farther down the tree?
8     Transplant (T, y, y.right) // replace y by its right child
9     y.right = z.right           // z's right child becomes
10    y.right.parent = y          // y's right child
11 Transplant (T, z, y)            // replace z by its successor y
12 y.left = z.left                 // and give z's left child to y,
13 y.left.parent = y              // which had no left child
```



Trace of TreeDelete (T, 13)

Implementation

Node for Binary Search Tree

Algorithm 2: Node for Binary Search Tree

```

1 begin
2   data
3   parent           // reference to the parent
4   left             // reference to the left child
5   right            // reference to the right child

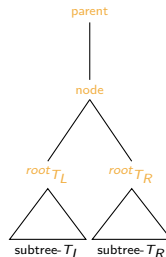
```

[[2]]

In **binary search tree**, each node has 0, 1 or 2 subtrees.

Use node with three pointers:

- **parent**: points the parent
- **left**: points the left child
- **right**: points the right child



Binary Search Tree property:

- Every node in subtree T_L is smaller than **node**
- Every node in subtree T_R is larger than **node**

Q. Is **node.data** smaller than **parent.data**?

Q. Is **node.data** larger than **parent.data**?

Assume that data at each node is different.

MyNodeBST

Algorithm 3: Node for Binary Search Tree

```

1 begin
2   data
3   parent           // reference to the parent
4   left             // reference to the left child
5   right            // reference to the right child

```

[[2]]

MyNodeBST implements interface NodeBSTInterface.

- T implements Comparable

$$y.data.compareTo(x.data) = \begin{cases} < 0, & y.data < x.data, \\ 0, & y.data = x.data, \\ > 0, & y.data > x.data. \end{cases}$$

```

1 MyNodeBST<T> x = ...;
2 MyNodeBST<T> y = ...;
3
4 // compare x and y
5 if (y.data.compareTo(x.data) < 0) {
6   // y is smaller than x
7 } else {
8   // y is larger than x
9 }

```

```

1 public class MyNodeBST<T extends Comparable<T>>
2   implements MyNodeBSTInterface<T> {
3   T data;
4   MyNodeBST<T> parent;
5   MyNodeBST<T> left;
6   MyNodeBST<T> right;
7
8   public MyNodeBST() {
9     this(null);
10  }
11  public MyNodeBST(T data) {
12    this.data = data;
13    parent = null;
14    left = null;
15    right = null;
16  }
17  ...
18 }

```

```

1 public interface NodeBSTInterface<T> {
2
3   T data();
4
5   MyNodeBSTInterface<T> parent();
6
7   MyNodeBSTInterface<T> left();
8
9   MyNodeBSTInterface<T> right();
10
11  T canonical();
12
13 }

```

MyBST, constructor

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3
4     public MyBST() {
5         root = null;
6     }
7     public void insert(T data) {
8         if (data == null) {
9             return;
10        }
11        MyNodeBST<T> x = root;
12        MyNodeBST<T> y = null;
13        MyNodeBST<T> z = new MyNodeBST<>(data);
14        while (x != null) {
15            y = x;
16            if (z.data.compareTo(x.data) < 0) {
17                x = x.left;
18            } else {
19                x = x.right;
20            }
21        }
22        // location is found.
23        z.parent = y;
24        if (y == null) {
25            root = z;
26        } else if (z.data.compareTo(y.data) < 0) {
27            y.left = z;
28        } else {
29            y.right = z;
30        }
31    }
32    ...
33 }
```

```

1 public class MyNodeBST<T extends Comparable<T>>
2     implements MyNodeBSTInterface<T> {
3     T data;
4     MyNodeBST<T> parent;
5     MyNodeBST<T> left;
6     MyNodeBST<T> right;
7
8     public MyNodeBST() {
9         this(null);
10    }
11    public MyNodeBST(T data) {
12        this.data = data;
13        parent = null;
14        left = null;
15        right = null;
16    }
17    ...
18 }
```

```

1 public class MyBSTConstructor {
2
3     public static MyBST<String>
4         constructBST_S_412_nodeByNode() {
5         MyBST<String> tree = new MyBST<>();
6         tree.insert("4");
7         tree.insert("1");
8         tree.insert("2");
9         return tree;
10    }
11    ...
12 }
```

MyBST, insert

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3
4     public MyBST() {
5         root = null;
6     }
7     public void insert(T data) {
8         if (data == null) {
9             return;
10        }
11        MyNodeBST<T> x = root;
12        MyNodeBST<T> y = null;
13        MyNodeBST<T> z = new MyNodeBST<>(data);
14        while (x != null) {
15            y = x;
16            if (z.data.compareTo(x.data) < 0) {
17                x = x.left;
18            } else {
19                x = x.right;
20            }
21        }
22        // location is found.
23        z.parent = y;
24        if (y == null) {
25            root = z;
26        } else if (z.data.compareTo(y.data) < 0) {
27            y.left = z;
28        } else {
29            y.right = z;
30        }
31    }
32    ...
33 }
```

```

1 public class MyBSTConstructor_Test {
2
3     private static final boolean DEBUG = true;
4
5     public static void main(String[] args) {
6         MyBST<String> tree;
7
8         tree = MyBSTConstructor.
9             constructBST_S_412_nodeByNode();
10        //
11        if (DEBUG) {
12            tree.plot();
13            System.out.println(tree.canonical());
14        }
15    }
16 }
```

```

1 public class MyBSTConstructor {
2
3     public static MyBST<String>
4         constructBST_S_412_nodeByNode() {
5         MyBST<String> tree = new MyBST<>();
6         tree.insert("4");
7         tree.insert("1");
8         tree.insert("2");
9         return tree;
10    }
11    ...
12 }
```

```
>4
  /2
 \1
/[1]/[2]\[4]\
```

MyBST, insert

```
1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3
4     public MyBST() {
5         root = null;
6     }
7     public void insert(T data) {
8         if (data == null) {
9             return;
10        }
11        MyNodeBST<T> x = root;
12        MyNodeBST<T> y = null;
13        MyNodeBST<T> z = new MyNodeBST<>(data);
14        while (x != null) {
15            y = x;
16            if (z.data.compareTo(x.data) < 0) {
17                x = x.left;
18            } else {
19                x = x.right;
20            }
21        }
22        // location is found.
23        z.parent = y;
24        if (y == null) {
25            root = z;
26        } else if (z.data.compareTo(y.data) < 0) {
27            y.left = z;
28        } else {
29            y.right = z;
30        }
31    }
32    ...
33 }
```

```
1 public class MyBSTConstructor_Test {
2
3     private static final boolean DEBUG = true;
4
5     public static void main(String[] args) {
6         MyBST<String> tree;
7
8         tree = MyBSTConstructor.
9             constructBST_S_412_nodeByNode();
10        //
11        if (DEBUG) {
12            tree.plot();
13            System.out.println(tree.canonical());
14        }
15    }
16 }
```

```
1 public class MyBSTConstructor {
2
3     public static MyBST<String>
4         constructBST_S_412_nodeByNode() {
5         MyBST<String> tree = new MyBST<>();
6         tree.insert("4");
7         tree.insert("1");
8         tree.insert("2");
9         return tree;
10    }
11    ...
12 }
```

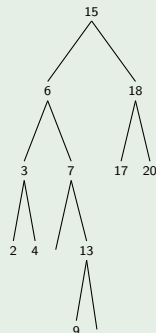
MyBST, search

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public MyNodeBST<T> successor() {
5         return (MyNodeBST<T>) LibTreeBS.successor(
6             root);
7     }
8     public MyNodeBST<T> successor(MyNodeBST<T> x)
9     {
10         return (MyNodeBST<T>) LibTreeBS.successor(x);
11     }
12     ...
13 }

```

Example



tree.search(13) ?
tree.search(4) ?
tree.search(5) ?

MyBST, search

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public MyNodeBST<T> successor() {
5         return (MyNodeBST<T>) LibTreeBS.successor(
6             root);
7     }
8     public MyNodeBST<T> successor(MyNodeBST<T> x)
9     {
10         return (MyNodeBST<T>) LibTreeBS.successor(x)
11         ;
12     }
13     ...
14 }

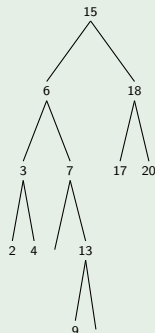
```

```

1 search(13): 15 → 6 → 7 → 13
2 search(4): 15 → 6 → 3 → 4
3 search(5): 15 → 6 → 3 → 4 → not found

```

Example



```

tree.search(13) ?
tree.search(4) ?
tree.search(5) ?

```

MyBST, minimum

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4
5     public MyNodeBST<T> minimum() {
6         return (MyNodeBST<T>) LibTreeBS.minimum(root);
7     }
8
9     public MyNodeBST<T> minimum(MyNodeBST<T> x) {
10        return (MyNodeBST<T>) LibTreeBS.minimum(x);
11    }
12    ...
13 }

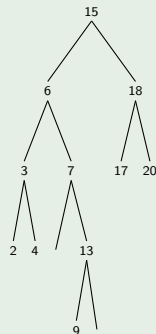
```

```

1 public class LibTreeBS<T extends Comparable<T>>
2 {
3     ...
4     public static <T> NodeBSTInterface<T> minimum(
5         NodeBSTInterface<T> x
6     ) {
7         while (x.left() != null) {
8             x = x.left();
9         }
10        return x;
11    }
12    ...
13 }

```

Example



tree.minimum() ?
 tree.minimum(tree.root.right) ?
 tree.minimum(tree.root.right.right) ?

MyBST, minimum

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4
5     public MyNodeBST<T> minimum() {
6         return (MyNodeBST<T>) LibTreeBS.minimum(root);
7     }
8
9     public MyNodeBST<T> minimum(MyNodeBST<T> x) {
10        return (MyNodeBST<T>) LibTreeBS.minimum(x);
11    }
12    ...
13 }

```

```

1 public class LibTreeBS<T extends Comparable<T>>
2 {
3     ...
4     public static <T> NodeBSTInterface<T> minimum(
5         NodeBSTInterface<T> x
6     ) {
7         while (x.left() != null) {
8             x = x.left();
9         }
10        return x;
11    }
12    ...
13 }

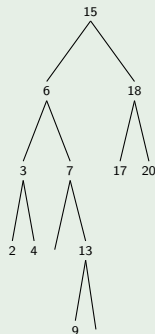
```

```

1 minimum(15): 2
2 minimum(18): 17
3 minimum(20): 20

```

Example



tree.minimum() ?
 tree.minimum(tree.root.right) ?
 tree.minimum(tree.root.right.right) ?

MyBST, maximum

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public MyNodeBST<T> maximum() {
5         return (MyNodeBST<T>) LibTreeBS.maximum(root);
6     }
7
8     public MyNodeBST<T> maximum(MyNodeBST<T> x) {
9         return (MyNodeBST<T>) LibTreeBS.maximum(x);
10    }
11    ...
12 }

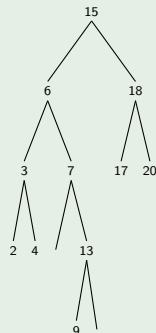
```

```

1 public class LibTreeBS<T extends Comparable<T>> {
2     ...
3     public static <T> NodeBSTInterface<T> maximum(
4         NodeBSTInterface<T> x
5     ) {
6         while (x.right() != null) {
7             x = x.right();
8         }
9         return x;
10    }
11    ...
12 }

```

Example



tree.maximum() ?
 tree.maximum(tree.root.left) ?
 tree.maximum(tree.root.left.left) ?

MyBST, maximum

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public MyNodeBST<T> maximum() {
5         return (MyNodeBST<T>) LibTreeBS.maximum(root);
6     }
7
8     public MyNodeBST<T> maximum(MyNodeBST<T> x) {
9         return (MyNodeBST<T>) LibTreeBS.maximum(x);
10    }
11    ...
12 }

```

```

1 public class LibTreeBS<T extends Comparable<T>> {
2     ...
3     public static <T> NodeBSTInterface<T> maximum(
4         NodeBSTInterface<T> x
5     ) {
6         while (x.right() != null) {
7             x = x.right();
8         }
9         return x;
10    }
11    ...
12 }

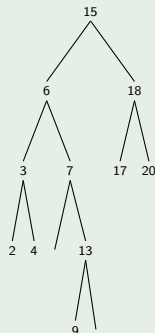
```

```

1 maximum(15): 20
2 maximum(6): 13
3 maximum(3): 4

```

Example



```

tree.maximum() ?
tree.maximum(tree.root.left) ?
tree.maximum(tree.root.left.left) ?

```

MyBST, successor

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public MyNodeBST<T> maximum() {
5         return (MyNodeBST<T>) LibTreeBS.maximum(root
6             );
7     }
8     public MyNodeBST<T> maximum(MyNodeBST<T> x) {
9         return (MyNodeBST<T>) LibTreeBS.maximum(x);
10    }
11    ...
12 }

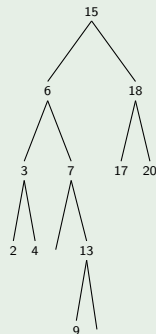
```

```

1 public class LibTreeBS<T extends Comparable<T>>
2 {
3     public static <T> NodeBSTInterface<T>
4         successor(NodeBSTInterface<T> x) {
5         if (x.right() != null) {
6             // leftmost node in the right subtree
7             return minimum(x.right());
8         } else {
9             // find the lowest ancestor of x whose
10             // left child is an ancestor of x
11             NodeBSTInterface<T> y = null;
12             y = x.parent();
13             while (y != null && x == y.right()) {
14                 x = y;
15                 y = y.parent();
16             }
17             return y;
18         }
19     }
20 }

```

Example



tree.successor() ?

tree.successor(tree.root.left) ?

tree.successor(tree.root.left.right.right) ?

MyBST, successor

```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public MyNodeBST<T> maximum() {
5         return (MyNodeBST<T>) LibTreeBS.maximum(root
6             );
7     }
8     public MyNodeBST<T> maximum(MyNodeBST<T> x) {
9         return (MyNodeBST<T>) LibTreeBS.maximum(x);
10    }
11    ...
12 }

```

```

1 public class LibTreeBS<T extends Comparable<T>>
2 {
3     public static <T> NodeBSTInterface<T>
4         successor(NodeBSTInterface<T> x) {
5         if (x.right() != null) {
6             // leftmost node in the right subtree
7             return minimum(x.right());
8         } else {
9             // find the lowest ancestor of x whose
10             // left child is an ancestor of x
11             NodeBSTInterface<T> y = null;
12             y = x.parent();
13             while (y != null && x == y.right()) {
14                 x = y;
15                 y = y.parent();
16             }
17             return y;
18         }
19     }
20 }

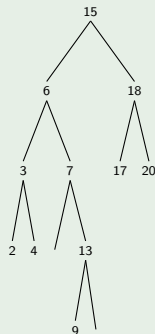
```

```

1 successor(15): 17
2 successor(6): 7
3 successor(13): 15

```

Example



tree.successor() ?

tree.successor(tree.root.left) ?

tree.successor(tree.root.left.right.right) ?

MyBST, delete (i)

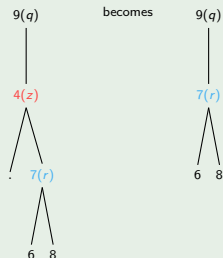
```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public void delete(MyNodeBST<T> z) {
5         MyNodeBST<T> y = null;
6         if (z.left == null) {
7             // replace z by its right child
8             transplant(z, z.right);
9         } else if (z.right == null) {
10            // replace z by its left child
11            transplant(z, z.left);
12        } else {
13            // y is z's successor
14            y = minimum(z.right);
15            if (y != z.right) {
16                // y is farther down in the tree
17                // replace y by its right child
18                transplant(y, y.right);
19                // z's right child becomes y's right
20                // child
21                y.right = z.right;
22                y.right.parent = y;
23            }
24            // replace z by its successor y
25            transplant(z, y);
26            // z's right child, which has no left child
27            // becomes y's right child
28            y.left = z.left;
29            y.left.parent = y;
30        }
31    }
32 }

```

Example

Node z has no left child.
We have $z < r$.



MyBST, delete (ii)

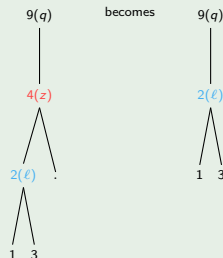
```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public void delete(MyNodeBST<T> z) {
5         MyNodeBST<T> y = null;
6         if (z.left == null) {
7             // replace z by its right child
8             transplant(z, z.right);
9         } else if (z.right == null) {
10            // replace z by its left child
11            transplant(z, z.left);
12        } else {
13            // y is z's successor
14            y = minimum(z.right);
15            if (y != z.right) {
16                // y is farther down in the tree
17                // replace y by its right child
18                transplant(y, y.right);
19                // z's right child becomes y's right
20                // child
21                y.right = z.right;
22                y.right.parent = y;
23            }
24            // replace z by its successor y
25            transplant(z, y);
26            // z's right child, which has no left child
27            // becomes y's right child
28            y.left = z.left;
29            y.left.parent = y;
30        }
31    }
32 }

```

Example

Node z has a left child ℓ but no right child.
We have $\ell < z$.



MyBST, delete (iii)

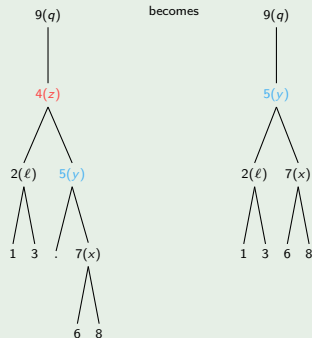
```

1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public void delete(MyNodeBST<T> z) {
5         MyNodeBST<T> y = null;
6         if (z.left == null) {
7             // replace z by its right child
8             transplant(z, z.right);
9         } else if (z.right == null) {
10            // replace z by its left child
11            transplant(z, z.left);
12        } else {
13            // y is z's successor
14            y = minimum(z.right);
15            if (y != z.right) {
16                // y is farther down in the tree
17                // replace y by its right child
18                transplant(y, y.right);
19                // z's right child becomes y's right
20                // child
21                y.right = z.right;
22                y.right.parent = y;
23            }
24            // replace z by its successor y
25            transplant(z, y);
26            // z's right child, which has no left child
27            // becomes y's right child
28            y.left = z.left;
29            y.left.parent = y;
30        }
31    }
32 }

```

Example

Node z has two children (ℓ and y). Node y is the successor of z , that is, y has no left child. Node x is the right child of y . Hence, $y = \text{successor}(z)$ and $\ell < z < y < x$.



MyBST, delete (iv)

```

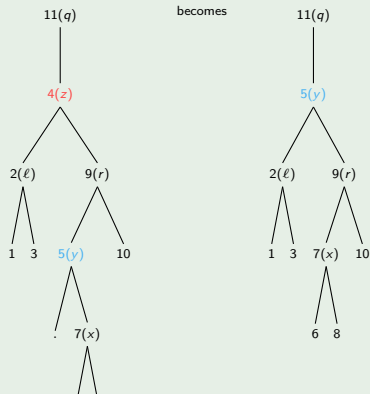
1 public class MyBST<T extends Comparable<T>> {
2     MyNodeBST<T> root;
3     ...
4     public void delete(MyNodeBST<T> z) {
5         MyNodeBST<T> y = null;
6         if (z.left == null) {
7             // replace z by its right child
8             transplant(z, z.right);
9         } else if (z.right == null) {
10            // replace z by its left child
11            transplant(z, z.left);
12        } else {
13            // y is z's successor
14            y = minimum(z.right);
15            if (y != z.right) {
16                // y is farther down in the tree
17                // replace y by its right child
18                transplant(y, y.right);
19                // z's right child becomes y's right
20                // child
21                y.right = z.right;
22                y.right.parent = y;
23            }
24            // replace z by its successor y
25            transplant(z, y);
26            // z's right child, which has no left child
27            // becomes y's right child
28            y.left = z.left;
29            y.left.parent = y;
30        }
31    }
32 }

```

Example

Node z has two children (ℓ and r) and its successor $y \neq r$ lies within the subtree rooted at r .

Hence, $y = \text{successor}(z)$ and $y \neq r$.



Applications

Some Applications of BST

Implement a set in math as a BST.

- Modify `insert` so that no duplication is allowed.
- If the BST is balanced, it would be fast to search, to insert and to delete in $O(\log n)$.

Database Management Systems (DBMS)

- DBMS keeps data in a system similar to BSTs.

References I

- [1] D. E. Knuth, *The Art of Computer Programming: Fundamental Algorithms, volume 1*, 2nd ed. Addison-Wesley Professional, 1973.
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms (CLRS)*, 4th ed. MIT Press, 2022.
- [3] B. Preiss, *Data Structures and Algorithms with Object-Oriented Design Patterns in Java*. Wiley, 1999.
- [4] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms*. Addison Wesley, 1974.
- [5] E. Horowitz and S. Sahni, *Fundamentals of Data Structures*. Pitman, 1982.
- [6] C. S. Horstmann, *Big Java: Early Objects*, 7th ed. John Wiley & Sons, 2019.
- [7] R. P. Grimaldi, *Discrete and Combinatorial Mathematics*, 5th ed. Pearson Education India, 2006.